AI-Based Fake QR Code Detection System with Vendor Verification and Weighted Risk Scoring

Teja Sri M
Dept. of AI and Data Science
RMK Engineering College
Tamil Nadu, India

Sowmiya M
Dept. of AI and Data Science
RMK Engineering College
Tamil Nadu, India

Supriya D
Dept. of AI and Data Science
RMK Engineering College
Tamil Nadu, India

***Abstract*—QR codes are widely used for digital payments and online services, but fake QR codes can redirect users to phishing websites or fraudulent vendors. This paper presents an AI-based fake QR code detection system that analyses a QR code before the user proceeds. The system decodes the payload, performs visual tamper analysis, classifies the embedded URL with a Random Forest model supported by Levenshtein-distance brand matching and Shannon-entropy checks, and, for UPI payment codes, verifies the vendor against registered merchant records. A weighted ensemble (30% image, 70% content) with penalty-based scoring produces a 0–100 risk score and a Safe, Suspicious, Vendor Mismatch or Malicious verdict. On a hold-out test set of 2,000 QR codes the model achieved 99.85% accuracy, 99.80% precision, 99.90% recall, a 99.85% F1-score and an ROC-AUC of 1.0, with inference latency below 50 ms. The system is implemented as a FastAPI backend with a Progressive Web App front end.**

***Index Terms*—QR code security, quishing, phishing detection, machine learning, Random Forest, vendor verification, UPI fraud, risk scoring.**

I. INTRODUCTION

Quick Response (QR) codes have become a primary interface for digital payments, ticketing, authentication and marketing. Their convenience also makes them an attractive attack surface: a malicious QR code can redirect a user to a phishing website, a credential-harvesting page or a fraudulent payment account, a threat commonly called *quishing*. Because the payload of a QR code is not human-readable, users cannot easily tell a genuine code from a malicious one before scanning it.

Existing countermeasures largely rely on checking the decoded URL against known blacklists [1], [2]. Such approaches struggle with new or unseen phishing links, ignore the visual characteristics of the QR image, and do not verify the identity of the vendor behind a payment code. Security checking and payment verification are usually handled as separate steps, so the user receives little real-time warning before paying.

This paper proposes an AI-based system that combines QR image analysis, URL analysis and vendor verification in a single pipeline and gives the user an immediate, explainable warning. The main contributions are: (i) a multi-stage detection pipeline that covers visual tamper, URL phishing and vendor mismatch risks; (ii) a weighted ensemble with transparent penalty-based scoring that yields a 0–100 risk score; (iii) a lightweight Progressive Web App implementation with sub-50 ms inference; and (iv) an evaluation on 2,000 unseen QR codes.

II. RELATED WORK

A. URL-Based Machine Learning Approaches

Several studies classify the URL embedded in a QR code. Secure QR scanners based on TF-IDF features with SVM and Random Forest classifiers were reported in [1], and a similar pipeline with threat-intelligence integration and anomaly detection in [2]. Amoah and Hayfron-Acquah extracted lexical URL features and found Logistic Regression best, with 96.47% accuracy [14]. Adaptive retraining with a BERT-based URL classifier reached 98.9% accuracy and an AUC of 0.999 on a Kaggle dataset [4], and a hybrid of TF-IDF, XGBoost and heuristic rules supported low-latency fraud detection [8].

B. Image and Deep Learning Approaches

Image-based work analyses the QR code itself. Trad and Chehab applied tree-based models directly to QR pixel structure, with XGBoost and LightGBM performing best (AUC up to 0.9133) [12]. A lightweight deep learning model with multi-objective optimisation reached 99% accuracy [6], and an AlexNet–PCA–GRU hybrid achieved 99.81% accuracy with 7.433 ms inference time [9]. A YOLOv8 and DenseNet121 pipeline for fake QR classification reported 92–92.5% accuracy [11]. ALFA targets fancy QR codes by converting them to binary grids and reports a false negative rate of 0.06% [3], while Adato detects adversarial QR code images at scale on social platforms [13].

C. Verification and Hybrid Frameworks

B-PhishQR uses AES-GCM encryption and blockchain-inspired integrity checks for deterministic verification without ML training [5]. Hybrid CNN, Vision Transformer and OCR frameworks were found to outperform traditional email filters at detecting QR phishing [7]. Most relevant to payments, a system combining CNN tamper detection, a BERT payload classifier and merchant verification achieved 96.7% accuracy with 340 ms scan-to-verdict latency [10].

D. Research Gaps

The literature shows that (a) most systems detect mainly known phishing links, (b) detection of new or unseen links is limited, (c) vendor identity is not sufficiently verified, (d) QR security and payment verification remain separate, and (e) real-time warnings before payment are limited. The proposed system addresses these gaps in one pipeline.

III. PROPOSED SYSTEM

A. Overview

Fig. 1 shows the architecture. The user scans a QR code with the mobile camera; the scanner detects and decodes it, the URL or payment details are extracted, and the input is cleaned and converted to features. The AI/ML detection module then runs three checks (visual tamper analysis, URL/phishing detection and vendor verification), and the risk analysis block combines their outputs into one score and verdict that is shown to the user.

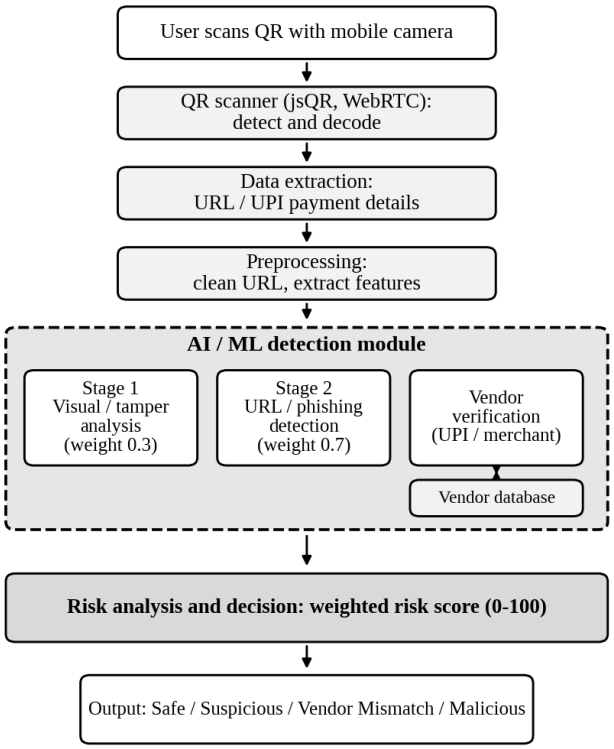


Fig. 1. Architecture of the proposed fake QR code detection system.

B. Visual and Tamper Analysis (Stage 1)

The QR image is checked for structural and visual irregularities such as a sticker pasted over a genuine code or altered modules. A detected physical overlay or tamper incurs a penalty of 40 points. The image result carries a weight of 0.3 in the final score.

C. URL and Content Analysis (Stage 2)

The decoded payload is analysed by a Random Forest classifier trained on URL features, together with rule-based checks. Brand impersonation and typosquatting (for example *amaz0n.com*) are found using Levenshtein distance against known brand domains. Domain names generated by bots are flagged when their Shannon entropy exceeds 3.8. Masked URL shorteners and insecure *http://* links are also penalised. Content analysis carries a weight of 0.7.

D. Vendor Verification

For UPI payment codes, the merchant name displayed in the code is compared with the registered vendor records. A code whose display name does not match the registered payment address is reported as Vendor Mismatch, and an unregistered individual address is reported as Suspicious.

E. Weighted Risk Scoring

Each scan starts at 100 points and penalties are subtracted for every risk found (Table I). The final score is the weighted ensemble

$$S_{final} = \text{round}(0.3 \cdot S_1 + 0.7 \cdot S_2) \quad (1)$$

where S_1 is the Stage 1 (image) score and S_2 is the Stage 2 (content) score.

TABLE I
RISK PENALTIES USED IN THE SCORING FORMULA

Detected risk	Penalty	Example
Physical sticker overlay / tamper	−40	Paper over stand
Brand impersonation / typosquatting	−55	amaz0n.com
Vendor mismatch (fraud account)	−45	Name ≠ VPA
Masked URL shortener	−35	bit.ly
Insecure HTTP protocol	−25	http://
High-entropy (DGA) domain, H > 3.8	−20	Random domain

IV. IMPLEMENTATION

The front end is a Progressive Web App built with HTML5, CSS3 and JavaScript (ES6+). QR scanning uses jsQR.js with the WebRTC and MediaDevices APIs and an HTML5 canvas, and scan history is kept in browser LocalStorage. The back end is written in Python with FastAPI and Uvicorn. Machine learning uses Scikit-learn, NumPy and Pandas, image handling uses Pillow, request validation uses Pydantic and Python TestClient, and the trained model is stored with Joblib. Matplotlib is used for result visualisation. Fig. 2 shows the home screen of the application, which offers a one-tap scan, scan statistics and demo attack scenarios.

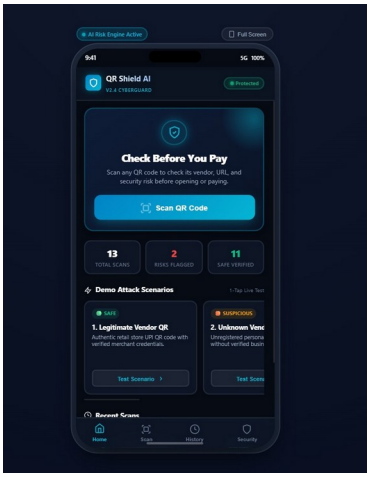


Fig. 2. Home screen of the QR Shield AI web application.

V. EXPERIMENTAL RESULTS

A. Dataset and Setup

The dataset of 10,000 QR code samples was split 80:20 into a training set of 8,000 codes and a hold-out test set of 2,000

codes that the model never saw during training. The test set contains 1,000 benign and 1,000 malicious QR codes.

B. Evaluation Metrics

Performance is measured with accuracy, precision, recall and F1-score, computed from the confusion matrix in Fig. 3:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (2)$$

$$Precision = TP / (TP + FP) \quad (3)$$

$$Recall = TP / (TP + FN) \quad (4)$$

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall) \quad (5)$$

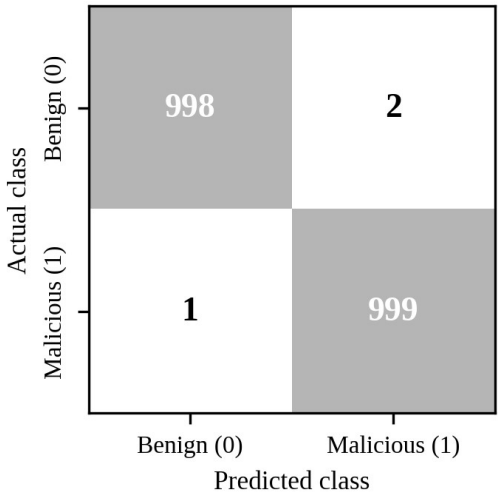


Fig. 3. Confusion matrix on the 2,000-sample hold-out test set.

The model correctly identified 999 of 1,000 malicious codes (TP) and 998 of 1,000 benign codes (TN), with 2 false positives and 1 false negative. This gives an accuracy of 1,997/2,000 = 99.85%, precision of 999/1,001 = 99.80% and recall of 999/1,000 = 99.90%. Recall is the most critical measure here, since a missed malicious code can lead directly to financial loss. Table II compares the results with the benchmarks we targeted.

TABLE II
PERFORMANCE ON THE HOLD-OUT TEST SET

Metric	Achieved	Benchmark	Remark
Accuracy	99.85%	> 95.0%	1,997 / 2,000
Precision	99.80%	> 94.0%	2 false alarms
Recall	99.90%	> 98.0%	1 attack missed
F1-score	99.85%	> 95.0%	Balanced
ROC-AUC	1.0000	> 0.980	Full separation
Inference latency	< 50 ms	< 200 ms	Real-time

C. Scenario Testing

Five demonstration scenarios were run through the application (Table III). The legitimate store scored 98/100 (Safe). An unregistered individual UPI address lost 35 points and was marked Suspicious. A vendor mismatch lost 45 points. The HDFC phishing portal lost 55 points for typosquatting and 25 for insecure HTTP, and the malicious shortener lost 35 for the masked redirect and 20 for high entropy, both ending at 32/100 (Malicious).

TABLE III
SCENARIO TEST RESULTS

Scenario	Penalties	Score / verdict
Legitimate ABC Store	None	98 / Safe
Unknown individual UPI	−35 (unregistered)	74 / Suspicious
Vendor mismatch fraud	−45 (name ≠ VPA)	55 / Mismatch
HDFC phishing portal	−55, −25	32 / Malicious
Malicious shortener	−35, −20	32 / Malicious

D. Comparison with Related Work

Table IV places the proposed system alongside accuracies reported in the literature. The figures come from different datasets and settings, so the comparison is indicative rather than a direct benchmark.

TABLE IV
COMPARISON WITH REPORTED ACCURACY

Ref.	Approach	Reported result
[14]	Lexical URL features + Logistic Regression	96.47% accuracy
[11]	YOLOv8 + DenseNet121	92–92.5% accuracy
[10]	CNN + BERT + merchant check	96.7% accuracy
[4]	BERT with adaptive retraining	98.9% accuracy
[6]	Lightweight deep learning	99% accuracy
[9]	AlexNet + PCA + GRU	99.81% accuracy
Ours	**RF + image/tamper + vendor check**	**99.85% accuracy**

E. Limitations

An AUC of 1.0 shows that benign and malicious samples are almost perfectly separable in our dataset, which will not always hold in the wild. Adversarial and stylised QR codes, new brands and a limited vendor database may reduce accuracy, so the system should be validated on larger real-world data before deployment.

VI. CONCLUSION AND FUTURE WORK

This paper presented an AI-based fake QR code detection system that unites visual tamper analysis, URL phishing detection and vendor verification and reports a clear Safe, Suspicious, Vendor Mismatch or Malicious verdict with a confidence score. It achieved 99.85% accuracy and 99.90% recall on 2,000 hold-out QR codes with latency under 50 ms. Future work includes adaptive retraining with fresh threat-intelligence feeds, evaluation on real-world and adversarial QR datasets, integration with UPI applications, and a native mobile client.

ACKNOWLEDGMENT

The authors thank the Department of Artificial Intelligence and Data Science, RMK Engineering College, for supporting this work carried out in the Product Development Laboratory.

REFERENCES

[1] "Secure QR code scanner to detect malicious URL using machine learning," 2023.

[2] “QR code malicious URL detection system using machine learning,” 2025.
[3] “ALFA: A safe-by-design approach to mitigate quishing attacks launched via fancy QR codes,” 2026.
[4] “QR code security: An adaptive retraining approach for dynamic URL-based threat detection,” 2026.
[5] “B-PhishQR: A blockchain-based framework for secure QR code verification against phishing attacks,” 2026.
[6] “Detection of QR code-based cyberattacks using a lightweight deep learning model,” 2024.
[7] Trivedi, Jangal and Gupta, “Phishing detection in advanced QR code attacks: Challenges and AI-driven solutions,” *IJRASET*, Jan. 2025.
[8] “AI based QR code fraud detection system,” 2025.
[9] “Efficient malicious QR code detection system using an advanced deep learning approach,” 2025.
[10] “AI-based QR code fraud and scam detection system for secure digital payments,” 2026.
[11] Supriya K V, “Deep learning-based detection and classification of fake QR codes,” *IJSREM*, Sept. 2025.
[12] Trad and Chehab, “Detecting quishing attacks with machine learning techniques through QR code analysis,” *arXiv*, 2025.
[13] Wu *et al.*, “Shielding QR codes: Unveiling the real-world illicit promotion behind adversarial QR codes,” in *Proc. USENIX Security Symp.*, 2026.
[14] Amoah and Hayfron-Acquah, “QR code security: Mitigating the issue of quishing (QR code phishing),” *Int. J. Comput. Appl.*, Oct. 2022.